

Advanced data analysis methods

PWF Summer school on Theoretical and Computational Biophysics

University of Colombo

Day 5/5

Dr. Vittorio Del Tatto

ML in a nutshell

- Machine learning: methods that learn to solve specific tasks from data, without explicit programming

Observations: “unlabelled” data $\{X_i\}$

Task: learn properties / features of $\{X_i\}$

Examples: dimensionality reduction, clustering

UNSUPERVISED
LEARNING

Observations: “labelled” data $\{X_i, y_i\}$

Task: learn the function $X \mapsto y$

Examples: classification, regression, feature selection

SUPERVISED
LEARNING

MACHINE
LEARNING

REINFORCEMENT
LEARNING

Observations: data generated through agent-environment interaction

Task: learn an optimal “policy” (state $\rightarrow$ action)

Examples: game playing, robot control

Input representation

- Independently of the method / task, we start from a dataset $\{X_i\}$ extracted from our simulation
 frame label (time index)
- X_i is a vector representing the state of the system at frame i and its components are called **features** (or **descriptors**)

Typically they characterize “geometric” properties (functions of the 3N positions)
- Choosing as features the 3N positions is not a good choice for two reasons:
 - not invariant under translation and rotations** of the simulation box
 if we apply a rigid global transformation we would like to keep the same representation (just a change of the reference frame!)
 - very **high-dimensional** vector
 it contains a lot of information that we don't need (e.g. the solvent configuration)
- Kinetics properties can be studied through the time evolution of the descriptors

Input representation – examples

- **Inter-atomic distances**, e.g. heavy-atom distances (if N heavy atoms, $N \times N$ matrix)
- **Contact maps**: matrix whose entry (i,j) is 1 if atoms i and j are “close”, 0 otherwise
- **Coordination numbers**: number of “close” atoms to a reference
- **Bond angles and dihedrals**, e.g. along the backbone of a peptide
- **Radius of gyration, end-to-end distance, RMSD wrt reference structure** (for peptides)
- **Local-order parameters** (e.g. distance to 5th neighbour, to describe local property around a ref atom)
- **Atom-centred descriptors**: list of rotational and translationally invariant features, e.g. ACSF (atom-centred symmetry functions) and SOAP (Smooth Overlap of Atomic Positions)
- **CV describing a transition (reaction coordinate)**, e.g. distance between binding pocket and ligand

Note that more descriptors could be used together, depending on properties we want to study (in this case, it's a good practice to **standardize different descriptors, especially if measured with different units**)

Feature selection vs dimensionality reduction

Feature selection

- Supervised: $\{X_i, y_i\}$
- Goal: find a **minimal subset of features** of which X_i is maximally informative of y_i



The output representation is **interpretable**

- Some methods: mutual information, regression trees

Dimensionality reduction

- Unsupervised: $\{X_i\}$
- Goal: find a **transformation of X_i onto a low-dimensional space** which retains as much information as possible



The output representation is (in general) **not interpretable**

- Some methods: next slide

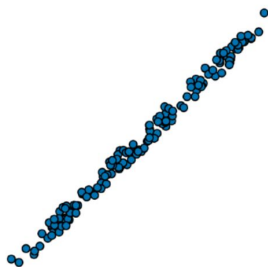
Dimensionality reduction

Goal: map X_i (typically high-dimensional) into a low-dimensional representation, for:

- Data visualization (2D, 3D)
- Using the low-dimensional representation as input of other analysis (supervised, unsupervised)
- Constructing CVs for enhanced sampling

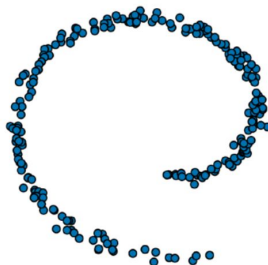
Methods:

Linear manifolds



- PCA
- MDS
- TICA

Curved-twisted manifolds



- Isomap
- Kernel-PCA
- Diffusion map

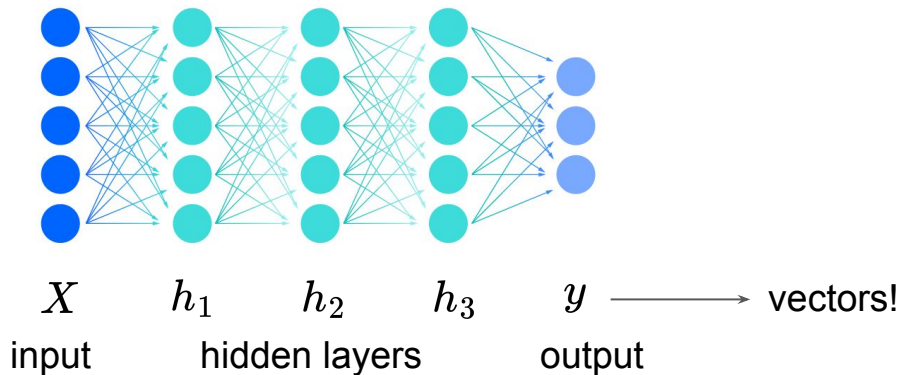
Highly nonlinear and complex manifolds



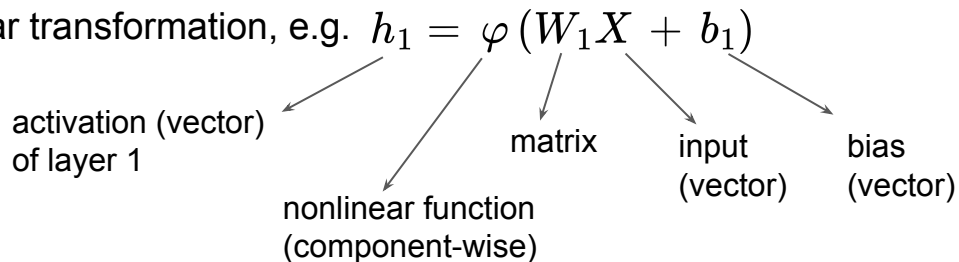
- sketchmap
- t-SNE, UMAP
- **Deep NNs**

Autoencoders

- First: a generic (feed-forward) Neural Network (FFNN)



- A NN is a sequence of operations which can be decomposed in a sequence of **layers**
- In each layer: a simple linear + nonlinear transformation, e.g. $h_1 = \varphi(W_1X + b_1)$



NNs in a nutshell

- Concatenating more layers:

$$y = \varphi \left(W_4 \varphi \left(W_3 \varphi \left(W_2 \varphi \left(W_1 X + b_1 \right) + b_2 \right) + b_3 \right) + b_4 \right)$$

The diagram illustrates the hierarchical structure of the neural network layers. Brackets are placed under the innermost expression $W_1 X + b_1$ and labeled h_1 . A larger bracket encompasses the expression $W_2 \varphi(h_1) + b_2$ and is labeled h_2 . The largest bracket encompasses the entire expression $W_4 \varphi(W_3 \varphi(W_2 \varphi(h_1) + b_2) + b_3) + b_4$ and is labeled h_3 .

- **Objective:** minimizing a **loss function**

→ e.g. if X has a continuous label $\hat{y}$ that we want to predict, a possible loss is

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N |\text{NN}(X_i) - \hat{y}_i|^2$$

depends on all the weights (and biases) in the network

NNs in a nutshell

- If we could compute the **gradient** of the loss function w.r.t. the weights, the problem would be solved!

$$\mathbf{w} \mapsto \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

learning rate

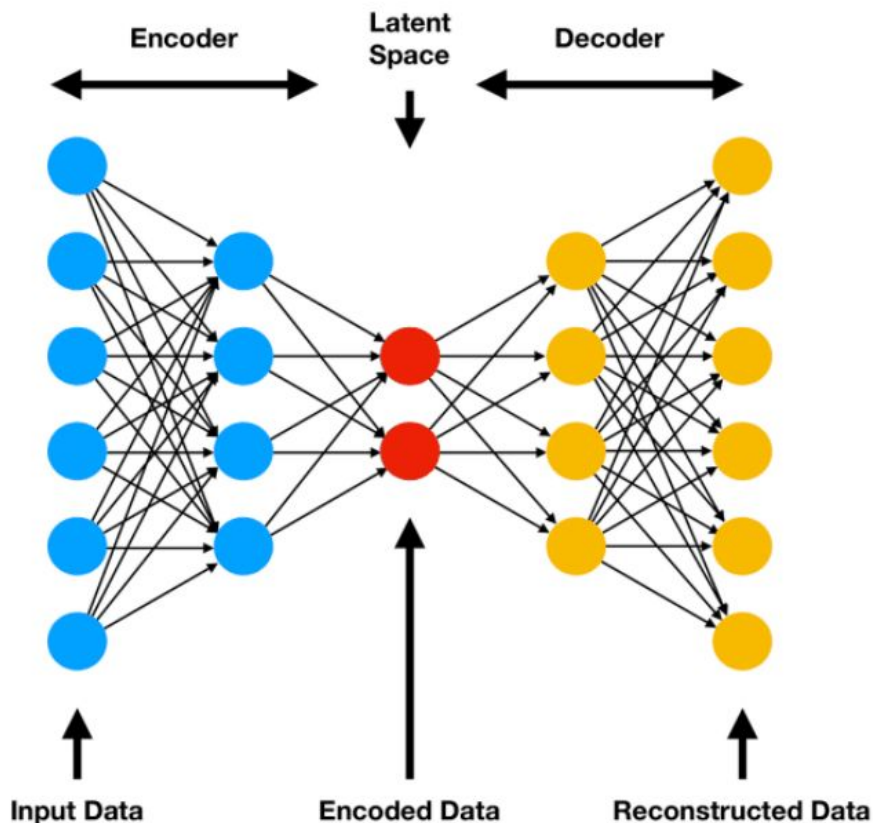
gradient descent rule, applied iteratively until convergence

- The gradient can be computed through **backpropagation**, an algorithm that implements the chain rule for derivatives iteratively (Hinton, 1986)

$$\frac{\partial}{\partial x} g(f(x)) = \frac{\partial g}{\partial f}(x) \frac{\partial f(x)}{\partial x}$$

$$\mathcal{L}(\mathbf{w}) \longrightarrow \left(\nabla_{W_4} \mathcal{L}(\mathbf{w}), \nabla_{b_4} \mathcal{L}(\mathbf{w}) \right) \longrightarrow \left(\nabla_{W_3} \mathcal{L}(\mathbf{w}), \nabla_{b_3} \mathcal{L}(\mathbf{w}) \right) \longrightarrow \dots$$

Autoencoders



$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^N \|\text{NN}(X_i) - X_i\|^2$$

Objective: reconstructing the input
(=learning the identity)

but passing through a
low-dimensional bottleneck

after training, the encoder can be
used to perform dimensionality
reduction

Limitation: NNs have lots of
parameters -> lots of data required!

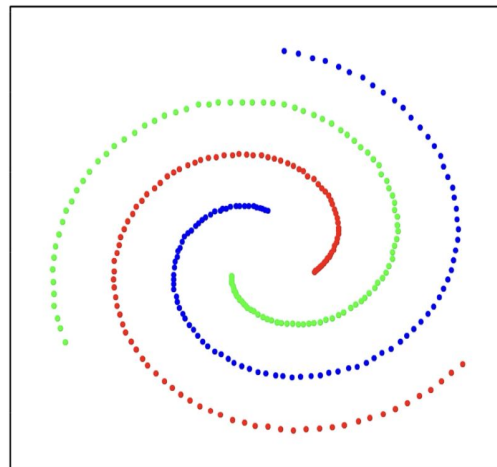
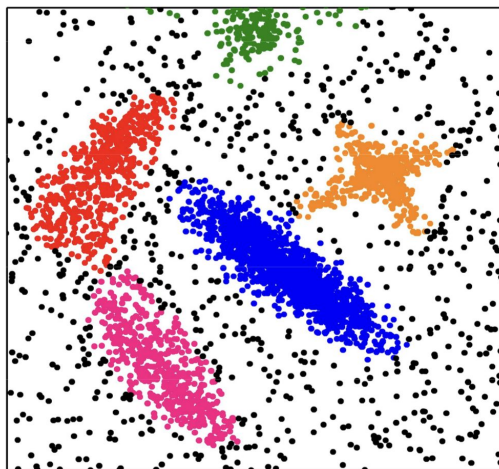
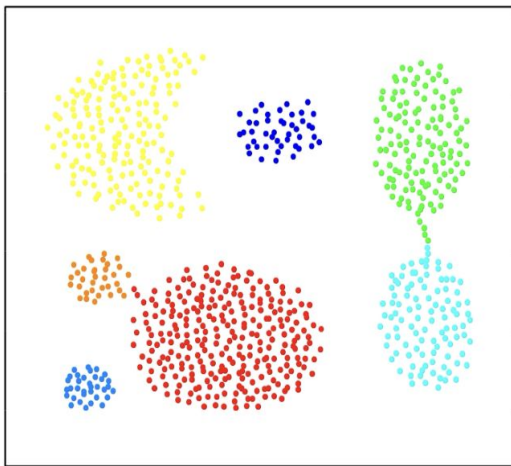
Clustering^{*}

- **Goal:** generating / discovering groups in the data



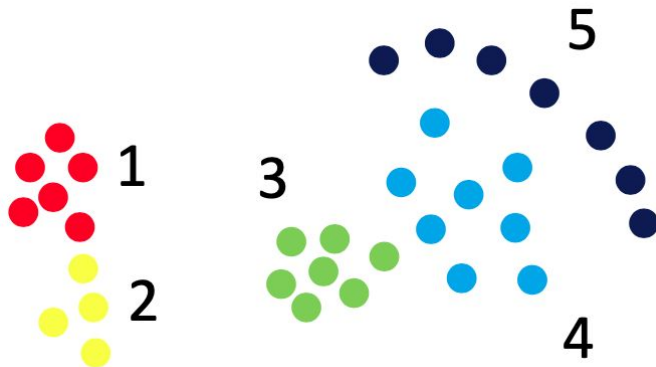
within molecular simulations, we'll see how this can be used to discover metastable states

- **What is a cluster?** Each method has its own answer, no single definition



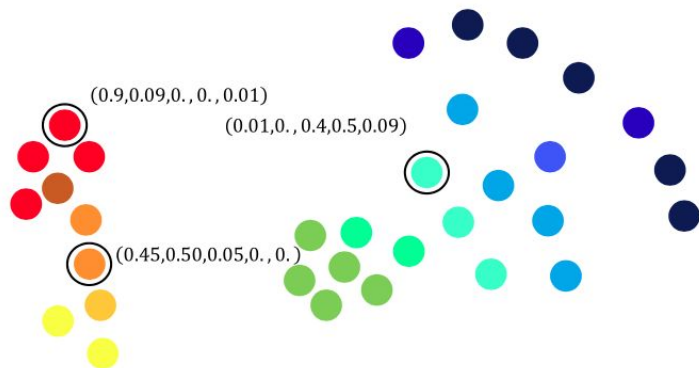
^{*} [material from Unsupervised Learning course held by Prof. Alessandro Laio and Prof. Alex Rodriguez, Trieste]

Flat clustering



- Each point assigned to a single cluster: $Cl(i) = l$
- Traditionally, the number of clusters (k) should be given as an external parameter
- Useful when one looks to a **hard partition**

Fuzzy clustering

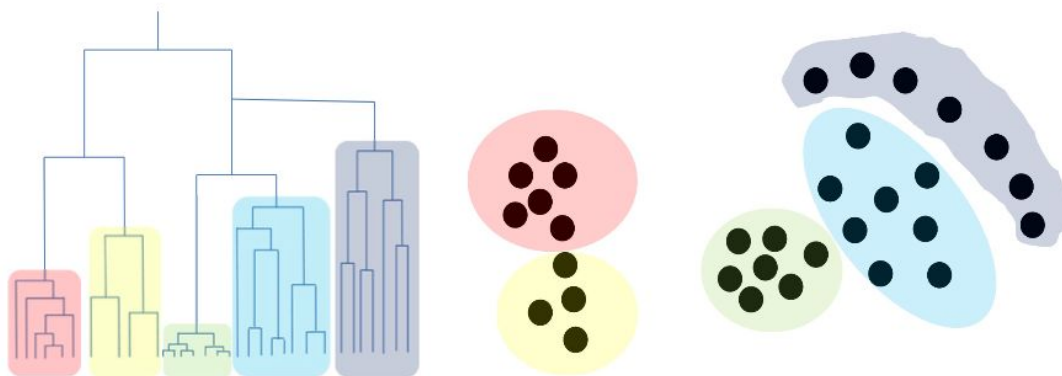


- Each point assigned to all clusters with a given degree of membership:

$$Cl(i) = (u_1, u_2, \dots, u_k) \quad \text{such that} \quad \sum_{i=1}^k u_i = 1$$

- Again, the number of clusters should be provided as an external parameter
- Provides a **soft partition**

Hierarchical clustering



- Produces a set of **nested clusters** organized as hierarchical tree (visualized as a **dendrogram**)
- No assumption on the number of clusters
- It can be transformed in many hard partitions depending on how the dendrogram is “cut” (“flattening” of the hierarchical clustering)

K-means

- J. MacQueen (1967); still widely used for its simplicity
- Flat clustering algorithm
- **Goal:** minimize intra-cluster distance while maximizing inter-cluster distance
- **Notation:** $Cl(i) = z_i$
- **Objective:**

$$\mathcal{L}(\{z_i\}) = \sum_{l=1}^k \sum_{i=1}^N \delta_{z_i, l} \|X_i - C_l\|^2$$

where C_l is the **centroid** of cluster l : $C_l = \frac{\sum_i \delta_{z_i, l} X_i}{\sum_i \delta_{z_i, l}}$

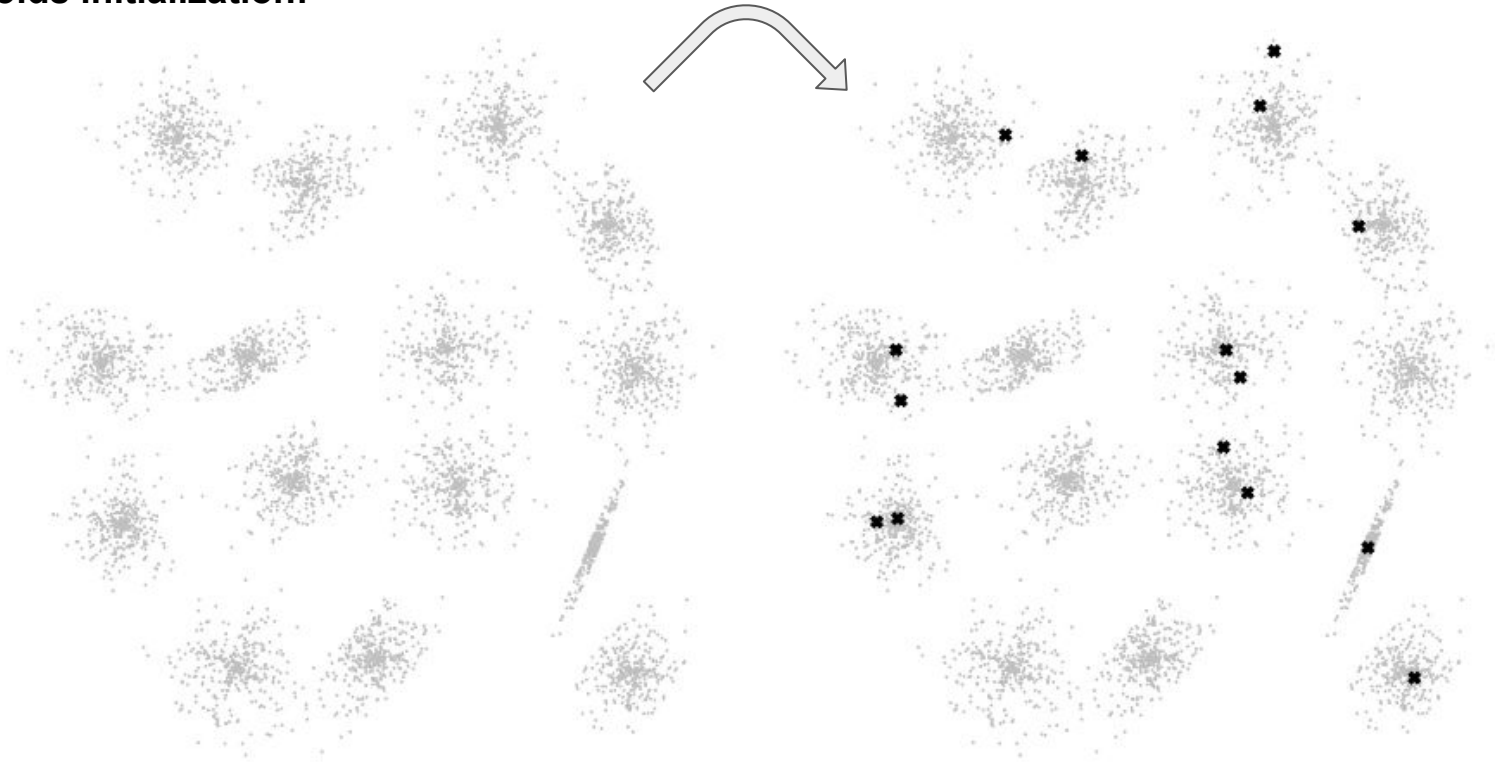
K-means

Algorithm

- **Input:** cluster size k , data points $\{X_i\}$
 - **Output:** cluster membership assignments $\{z_i\}$
1. Initialize k cluster centroids $\{C_l\}$ (randomly chosen from the N points)

K-means

Centroids initialization:



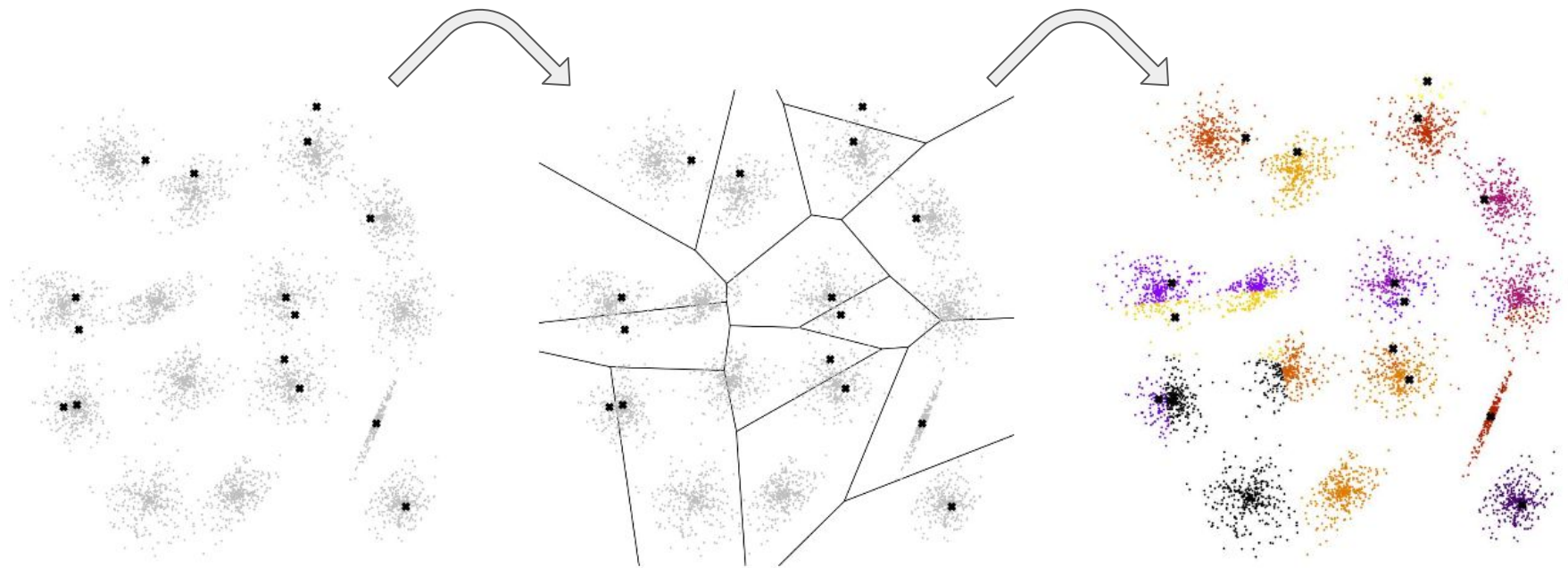
K-means

Algorithm

- **Input:** cluster size k , data points $\{X_i\}$
 - **Output:** cluster membership assignments $\{z_i\}$
1. Initialize k cluster centroids $\{C_l\}$ (randomly chosen from the N points)
 2. Repeat until convergence of all $\{z_i\}$:
 - a) assign z_i based on the closest centroid: $z_i = \operatorname{argmin}_l d(X_i, C_l)$

K-means

Membership assignment:



“Voronoi partition” (or “tessellation”)

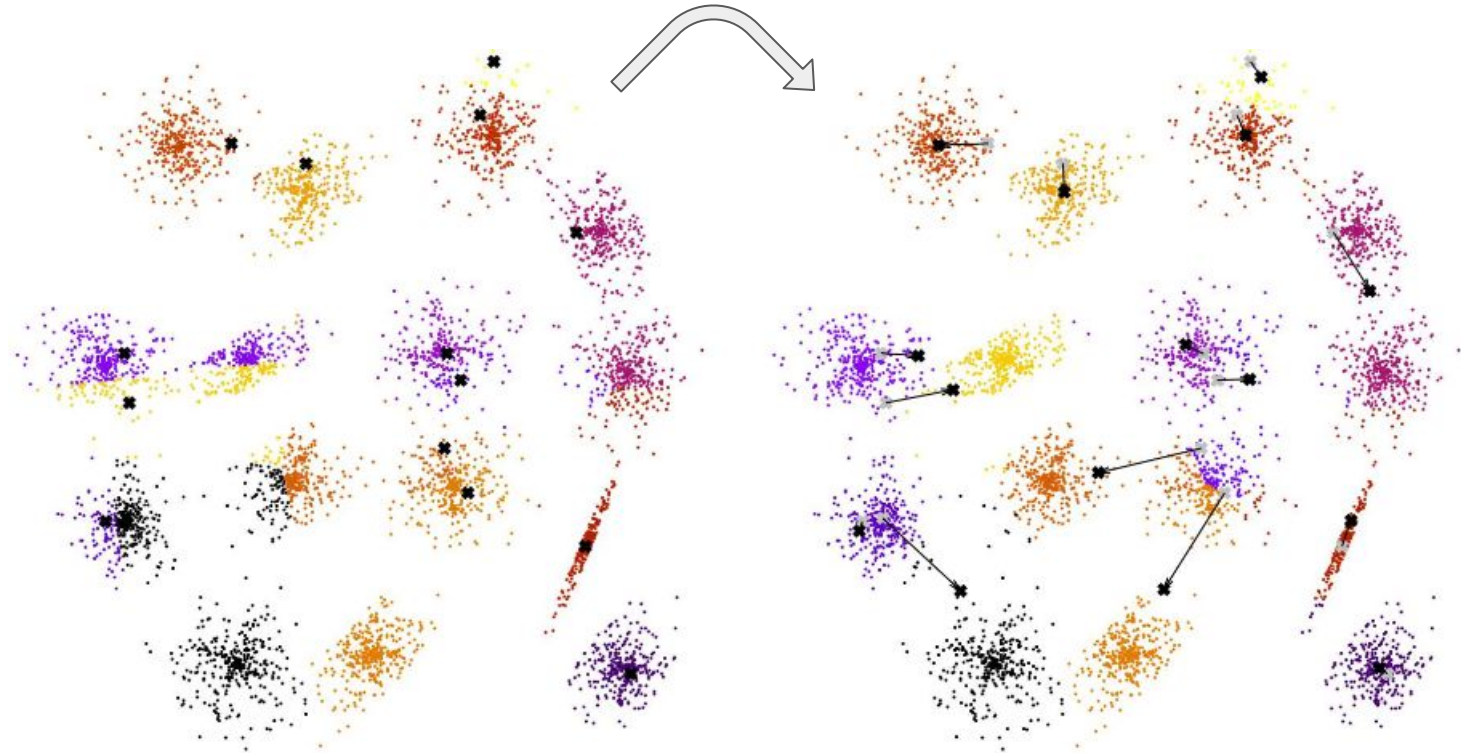
K-means

Algorithm

- **Input:** cluster size k , data points $\{X_i\}$
 - **Output:** cluster membership assignments $\{z_i\}$
1. Initialize k cluster centroids $\{C_l\}$ (randomly chosen from the N points)
 2. Repeat until convergence of all $\{z_i\}$:
 - a) assign z_i based on the closest centroid: $z_i = \operatorname{argmin}_l d(X_i, C_l)$
 - b) update the centroids based on the memberships:
$$C_l = \frac{\sum_i \delta_{z_i, l} X_i}{\sum_i \delta_{z_i, l}}$$

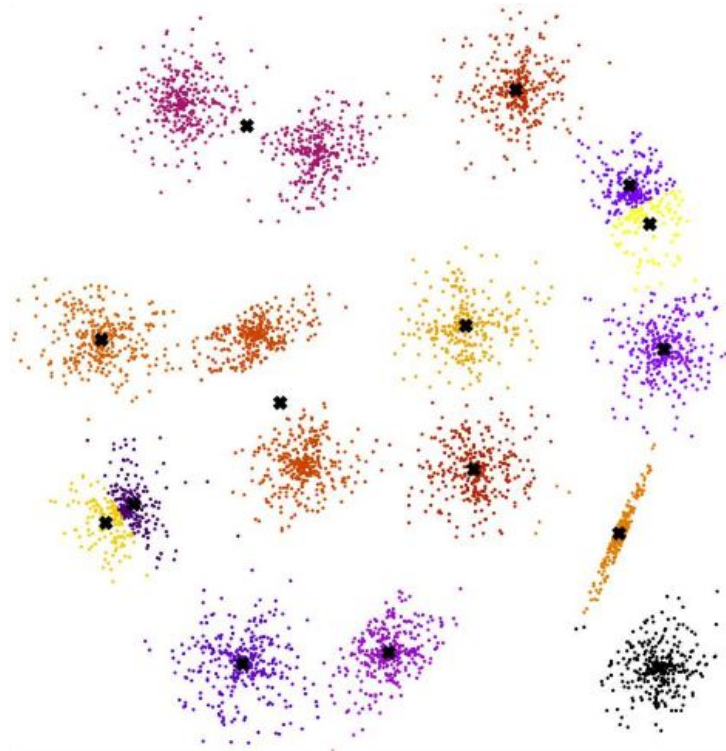
K-means

Centroid update:



K-means

After some iterations:



K-means

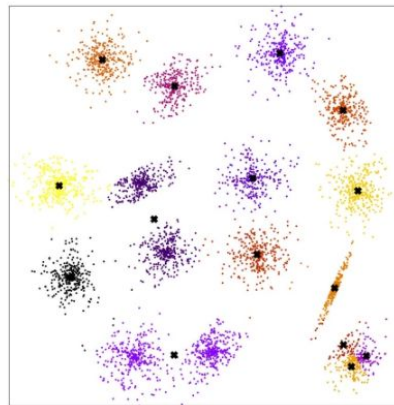
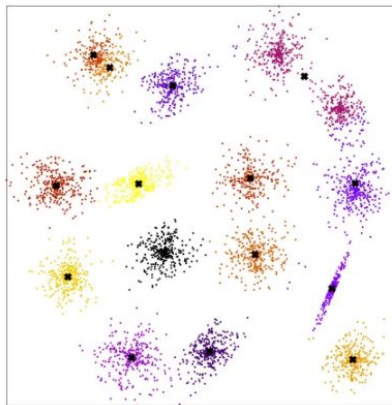
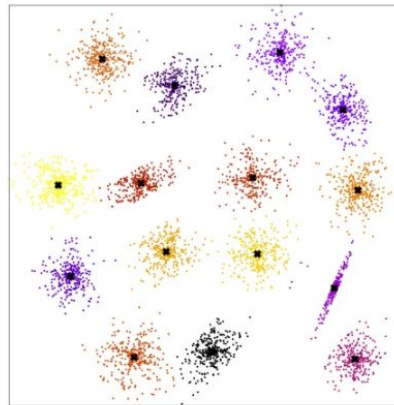
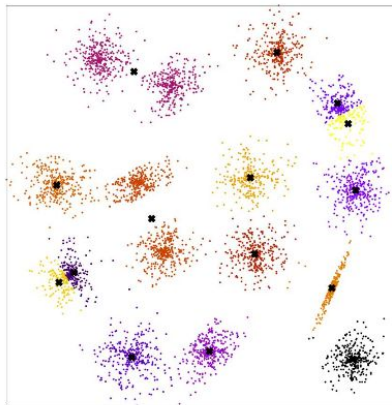
Weaknesses:

- **Sensitive to initialization** → k-means++
 - **k not specified** → “scree test”
 - **Sensitive to outliers**
 - **Based on Euclidean distance only**
 - **Only for spherical clusters** → “kernel k-means”
- } → “k-medoids”

K-means

Weaknesses:

- Sensitive to initialization:

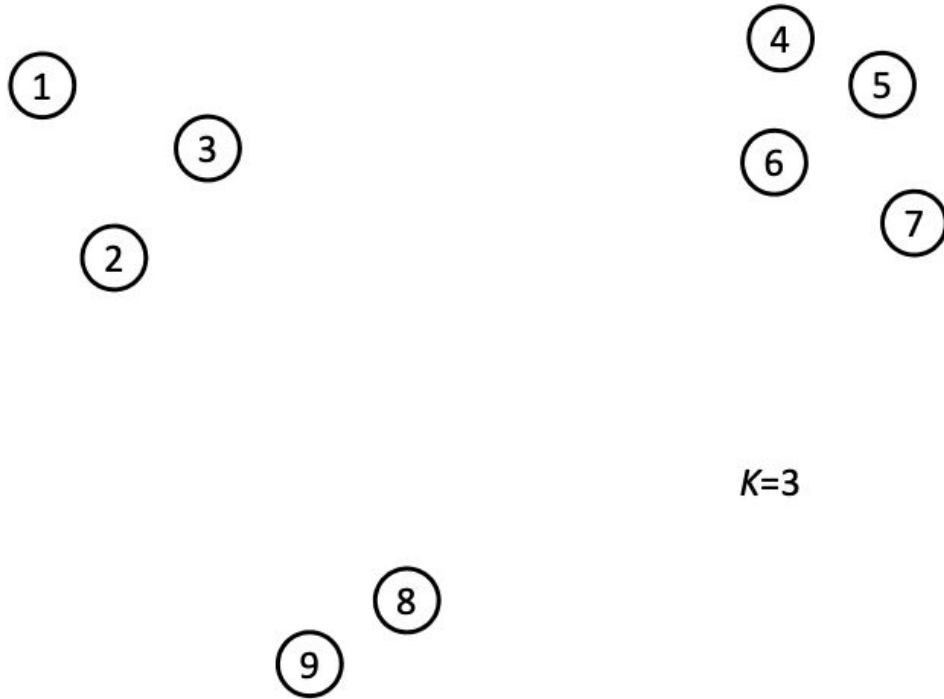


K-means++

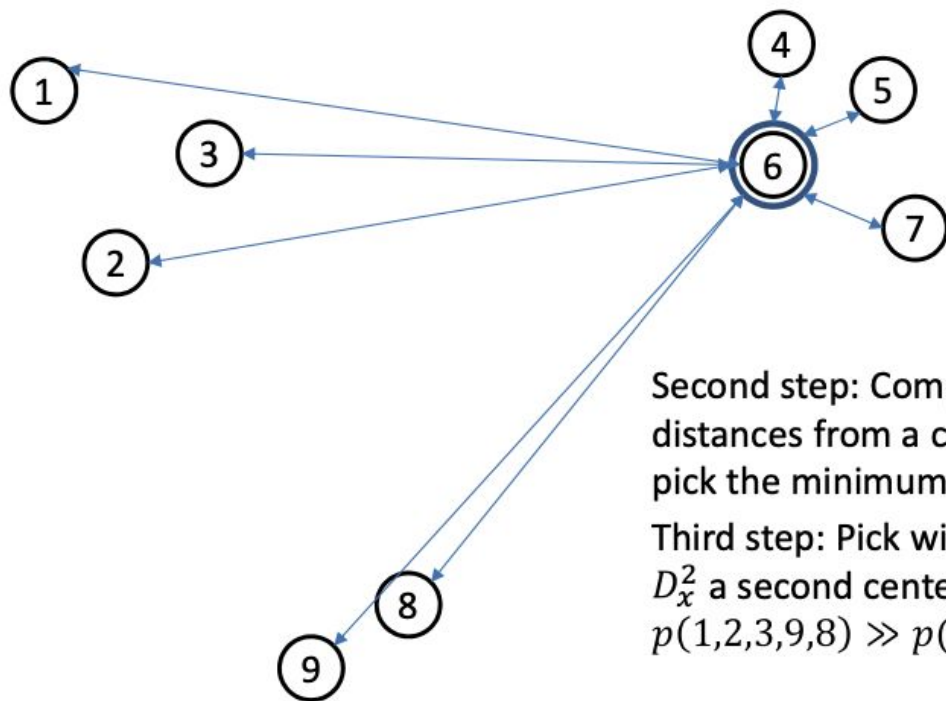
Better initialization:

1. Choose only the first centroid at random
2. Repeat until all k centers have been found:
 - For each point compute $D_i := \min_k d(X_i, C_k)$
 - Extract new centroid by sampling points with probabilities $P_i \propto D_i^2$
3. Run k-means with selected centroids as initialization

K-means++



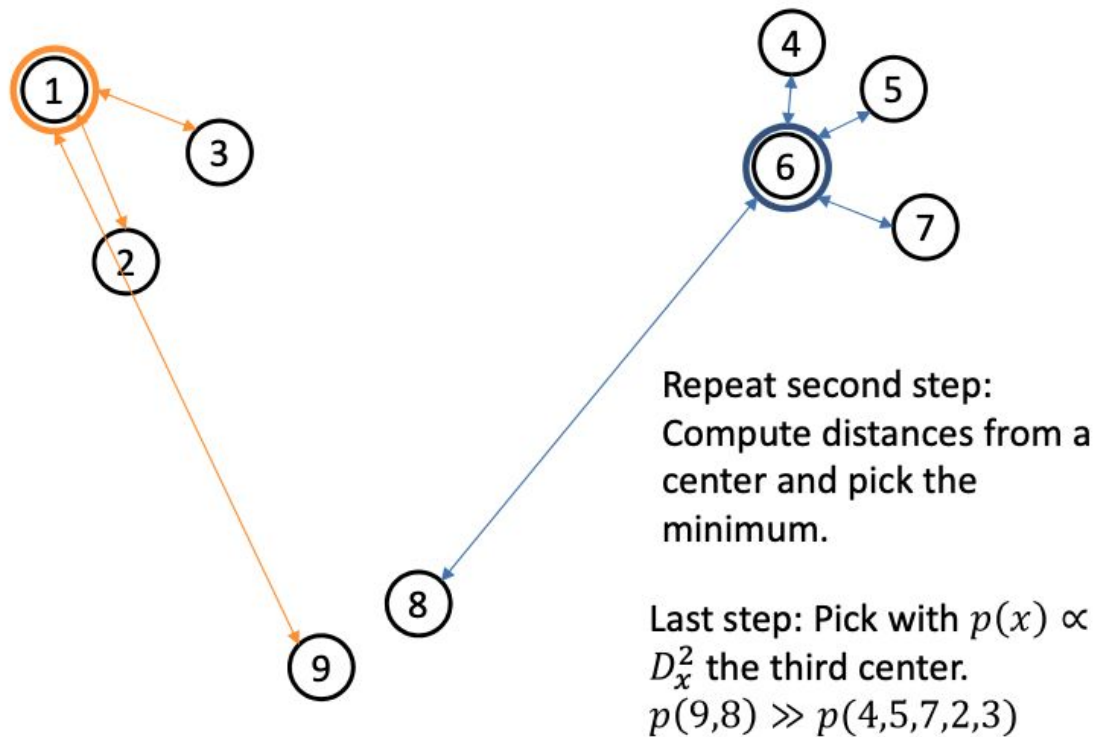
K-means++



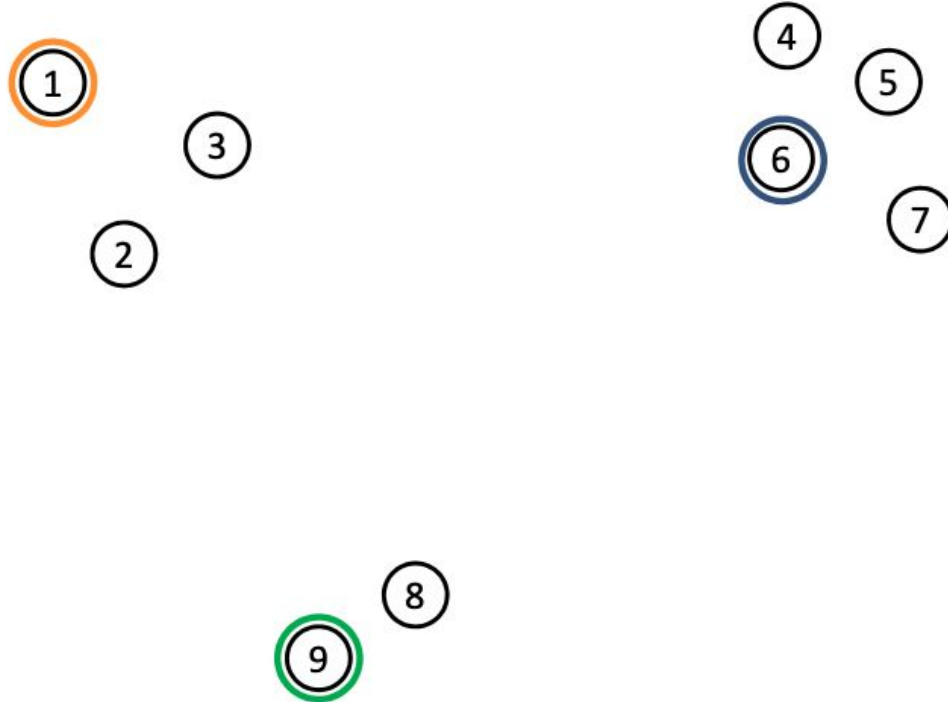
Second step: Compute distances from a center and pick the minimum.

Third step: Pick with $p(x) \propto D_x^2$ a second center.
 $p(1,2,3,9,8) \gg p(4,5,7)$

K-means++



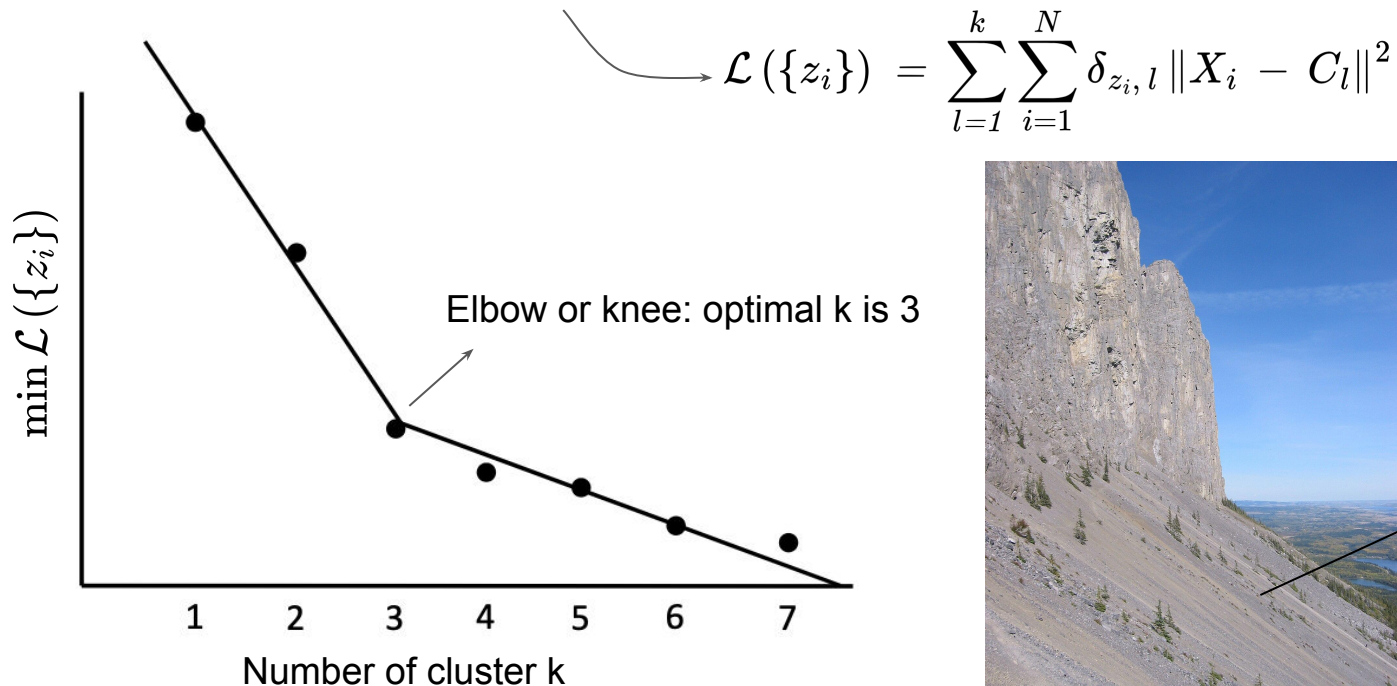
K-means++



Scree test

How many clusters k?

- Monitor the variation of the objective (at convergence) as a function of k:



Markov state modeling

- What we saw so far does not account for the **time series structure** of the data
 - results don't change if we permute the indices of $\{X_i\}$
- **Markov state modeling**: technique used to predict **long-term kinetic properties** of molecular systems (and other dynamic systems more in general)
- **Idea**: modelling the dynamics as jumps between macrostates (“Markov states”)
- **Goal**: extracting the **macrostates** associated to the slowest jumps and the corresponding **relaxation time scales**
 - ↓
from which transition times can be computed
 - ↘
= metastable states

Markov state modeling

- First step: **discretize** the space in which we are observing the dynamics
 This can be done through **clustering!** Each cluster will represent a **microstate**
- Notation: clusters denoted with labels $\alpha, \beta = 1, \dots, K$
- Second step: choose a lag $\mathcal{T}$ under which the dynamics is not “resolved” anymore

Assumption: the dynamics between the clusters is **Markovian**: the probability of finding the system in a given state at time $t + \tau$ only depends on the previous state at time t

We can use the Markov process theory learnt two days ago!

Markov state modeling

- **Review of the theory:** (new notation: states are denoted by α , β transition matrix describes jumps from t to $t + \tau$)
- Probability of finding the system in state α evolves according to the “**Master equation**”:

$$P_{\alpha}(t + \tau) = P_{\alpha}(t) + \underbrace{\sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha}}_{\text{“gain term”}} - \underbrace{\sum_{\beta} P_{\alpha}(t) \Pi_{\alpha\beta}}_{\text{“loss term”}}$$

Markov state modeling

$$P_{\alpha}(t + \tau) = P_{\alpha}(t) + \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} - \sum_{\beta} P_{\alpha}(t) \Pi_{\alpha\beta}$$

- Before going on: what are the differences w.r.t. what we were doing in day 3?
 - There we were using this theory for **sampling a target distribution**: P decided by us, Π built to satisfy stationarity
 - Here we don't want to sample: **trajectory was already generated and we want to analyze it**

↓

Π is given; we want to study its properties to understand better the kinetics in a “coarse-grained” space where the dynamics is Markovian

Markov state modeling

$$P_{\alpha}(t + \tau) = P_{\alpha}(t) + \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} - \sum_{\beta} P_{\alpha}(t) \Pi_{\alpha\beta}$$

- Let's simplify this expression in a different way w.r.t. day 3:

$$P_{\alpha}(t + \tau) = P_{\alpha}(t) + \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} - P_{\alpha}(t) \underbrace{\sum_{\beta} \Pi_{\alpha\beta}}$$

$$P_{\alpha}(t + \tau) = P_{\alpha}(t) + \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} - P_{\alpha}(t) = 1 \quad (\text{row-stochastic matrix!})$$

$$P_{\alpha}(t + \tau) = \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} \quad (\text{reformulation of the Master equation})$$

Markov state modeling

$$P_{\alpha}(t + \tau) = \sum_{\beta} P_{\beta}(t) \Pi_{\beta\alpha} \quad (\text{reformulation of the Master equation})$$

- Written in matrix form this becomes $P(t + \tau) \Pi = P(t)$
- The stationary probability P_{eq} therefore satisfies: $P_{\text{eq}} \Pi = P_{\text{eq}}$ **Stationarity condition**

It tells us that the stationary distribution is the **left eigenvector** of the transition matrix, with **eigenvalue equal to 1!**

Markov state modeling

Review of linear algebra:

- Given a matrix A , the vector v is called a **right eigenvector** of A with eigenvalue λ if

$$A v = \lambda v$$

- Given a matrix A , the vector u is called a **left eigenvector** of A with eigenvalue λ if

$$u^T A = \lambda u^T$$

↑
same form as:
↓

$$P_{\text{eq}} \Pi = P_{\text{eq}}$$

- If A is a square matrix (our case), the **left and right eigenvalues are the same!**
- In general, the eigenvalues are not necessarily real numbers: they can be **complex**
- The eigenvectors (left or right) form a **basis** of the space: each other vector can be written as a linear combination of eigenvectors

Markov state modeling

- What about the **other eigenvalues** of the transition matrix Π ?

↓

$$u_j^T \Pi = \lambda_j u_j^T$$

Perron-Frobenius theorem: if the dynamics generated by Π is ergodic, then:

- The stationary distribution is unique: the process converges to P independently of the initial conditions
- All the other eigenvalues $\{\lambda_j\}$ are smaller in modulus: $|\lambda_j| \leq 1$

Markov state modeling

- Let's exploit this property of the eigenvalues by expanding the solution of

$$P(t + \tau) = P(t) \Pi$$

over the set of left eigenvectors of Π

- Eigenvalues and left eigenvectors of Π :

$$\{\lambda_j\}, \{u_j\} \quad \text{with} \quad 1 = \lambda_0 < |\lambda_1| \leq |\lambda_2| \dots \leq |\lambda_K|$$

$$(u_0 = P_{\text{eq}})$$

- Expansion:
$$P(t) = \sum_{j=0}^K c^{(j)}(t) u^{(j)}$$

Markov state modeling

- Substituting the expansion in the equation describing the dynamics:

$$P(t) = P_{\text{eq}} + \sum_{j(>0)} c_j(0) \lambda_j^{t/\tau} u_j$$

- Let's reason on the exponentials $\lambda_j^{t/\tau}$: what is their behaviour for $t \rightarrow +\infty$?



Since they all have modulus smaller than 1, $\lambda_j^{t/\tau} \rightarrow 0$ for $t \rightarrow +\infty$

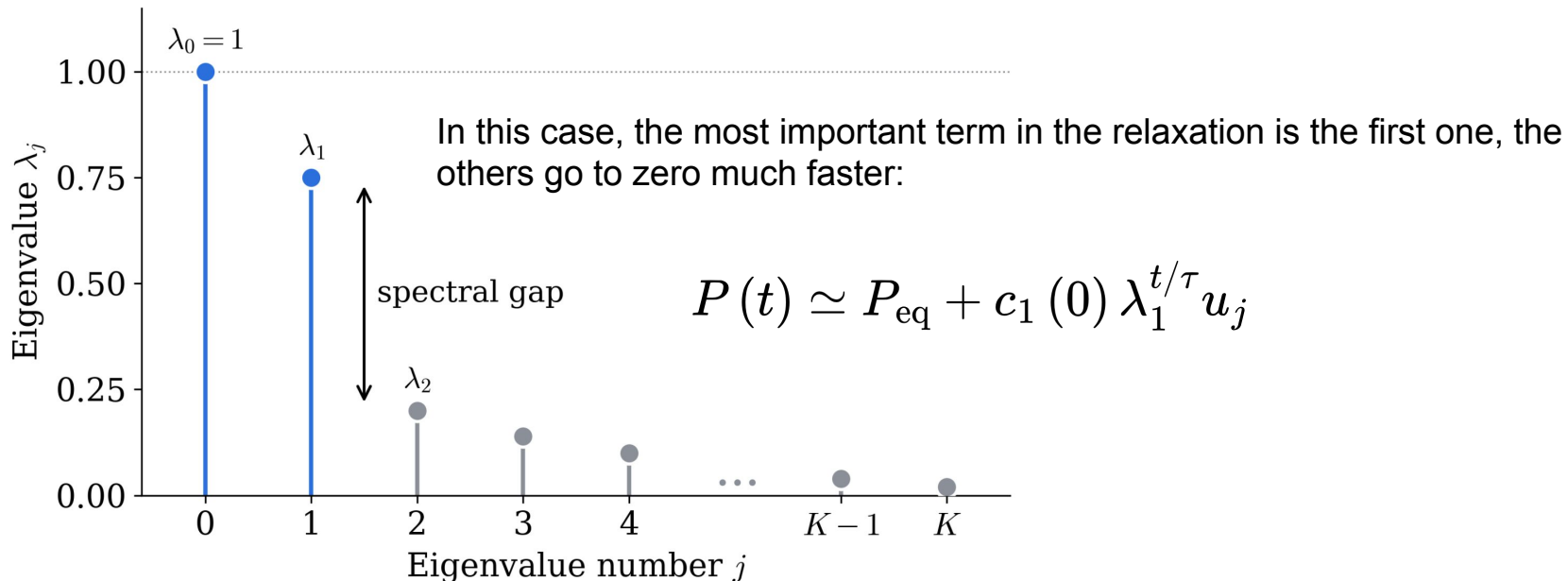


$$P(t) \rightarrow P_{\text{eq}} \quad \text{for} \quad t \rightarrow +\infty$$

Markov state modeling

Can we characterize the convergence to equilibrium?

- Let's suppose that the eigenvalue λ_1 has a modulus which is significantly larger than the others:



Markov state modeling

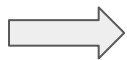
Can we characterize the convergence to equilibrium?

$$P(t) \simeq P_{\text{eq}} + c_1(0) \lambda_1^{t/\tau} u_1$$

1. $c_1(0)$ is just a constant which depends on the initialization
2. $\lambda_1^{t/\tau}$ defines the main relaxation time scale of the system

↓
relaxation timescales are defined through exponential decays:

$$\lambda_1^{t/\tau} = \exp\left(\log(\lambda_1^{t/\tau})\right) = \exp\left(\frac{t}{\tau} \log(\lambda_1)\right) = \exp\left(-\frac{t}{-\tau / \log |\lambda_1|}\right)$$



$$\tau_1 = -\frac{\tau}{\log |\lambda_1|}$$

Slowest relaxation timescale

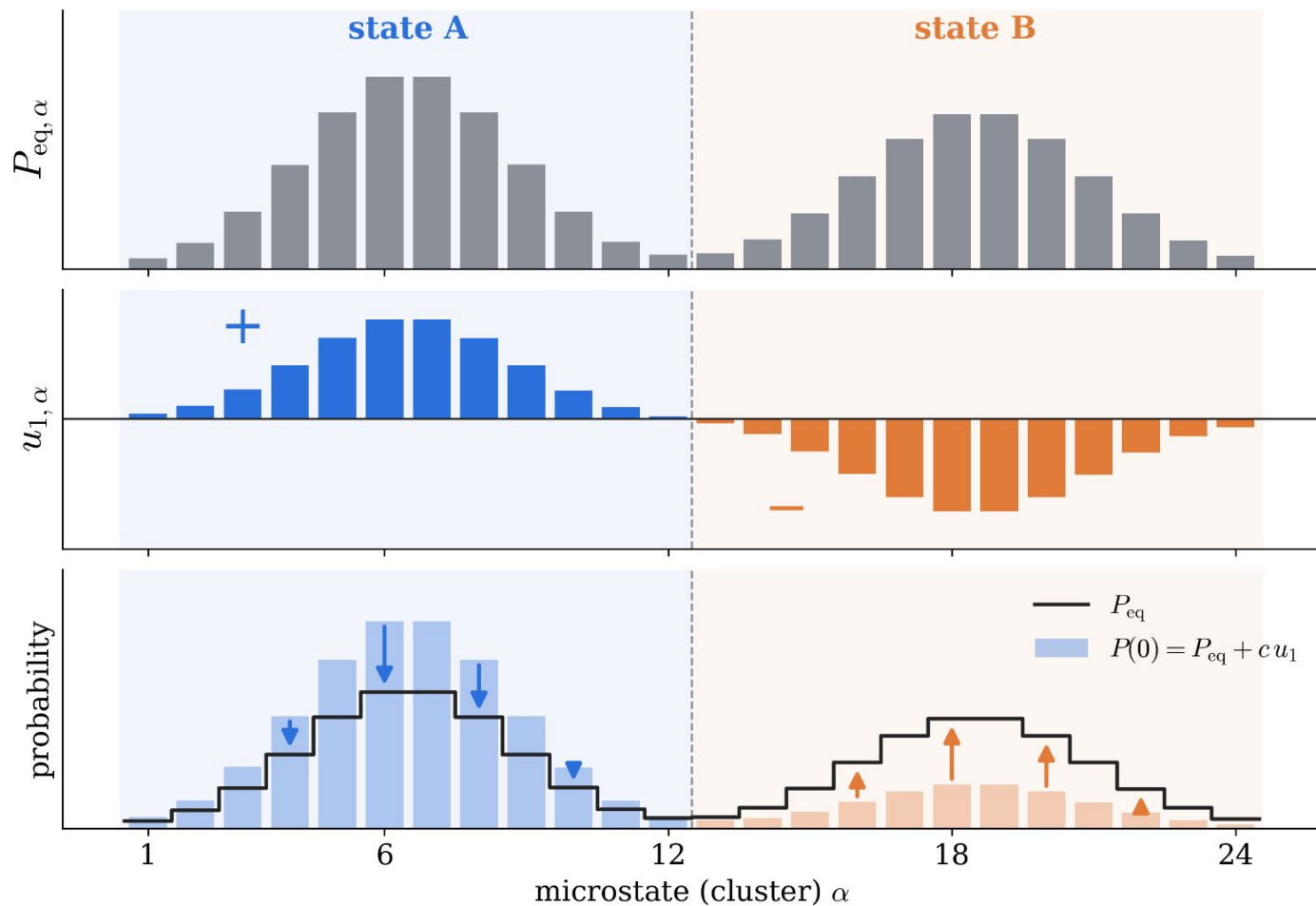
Markov state modeling

3. The **sign of the components** of the first eigenvector, $u_{1,\alpha}$, define the two main **Markov states** (or **metastable states**) of the system:

$$A := \{\alpha : u_{1,\alpha} > 0\}$$

$$B := \{\alpha : u_{1,\alpha} < 0\}$$

A **Markov state** is a set of microstates (=clusters) that converge to equilibrium from the same direction



Equilibrium distribution
($\lambda_0 = 1$)

Slowest mode
($|\lambda_1|$ close to 1)
 $\Sigma_{\alpha} u_{1, \alpha} = 0$

Relaxation
 $P(t) = P_{\text{eq}} + c \lambda_1^t u_1 + \dots$
 $\rightarrow P_{\text{eq}}$

Markov state modeling

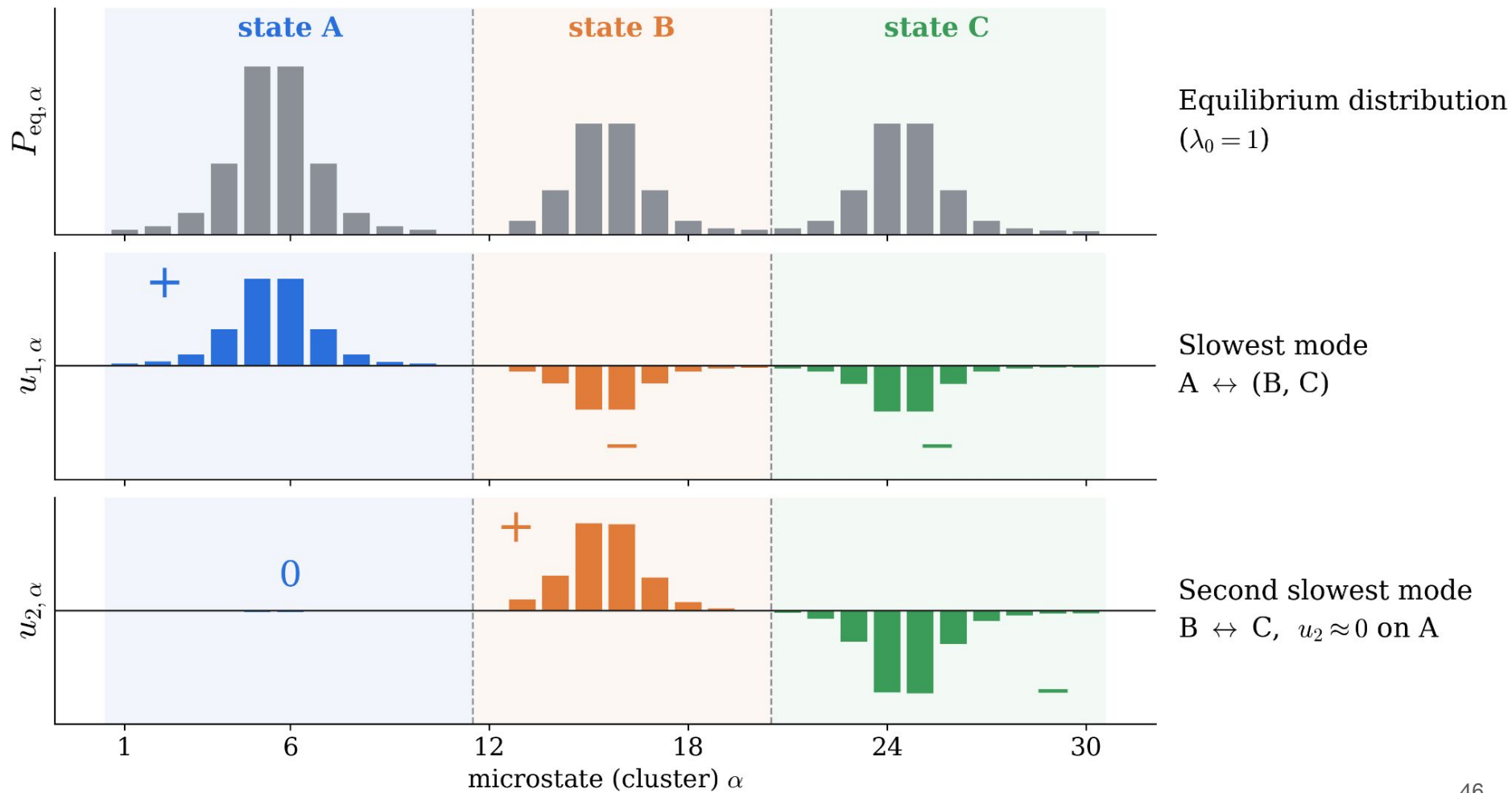
What if we have a gap after λ_2 ?

$$P(t) \simeq P_{\text{eq}} + c_1(0) \lambda_1^{t/\tau} u_1 + c_2(0) \lambda_2^{t/\tau} u_2$$

- **two** main relaxation timescales:

$$\tau_1 = -\frac{\tau}{\log |\lambda_1|} \quad \tau_2 = -\frac{\tau}{\log |\lambda_2|}$$

- **three** metastable states, looking at the joint signs of $u_{1,\alpha}$ and $u_{2,\alpha}$



Markov state modeling

What if we have no gap at all?

- No clear separation of timescales: the system has no set of states it stays in much longer than it takes to relax within them
- We can't identify clear Markov states and construct a Markov model
- N.B. this does not necessarily mean that the system does not have metastability: other possibilities are:
 - we used “bad CVs” for clustering (these CVs don't “see” the transitions)
 - the trajectory is too short to observe transitions (Markov state modeling needs ergodic trajectories from plain MD)

Markov state modeling

How to choose the hyperparameters?

- K (number of microstates=clusters): if we had infinite data, we would take a very large number of clusters


Why? Because discretizing the space is an approximation, whose error goes to zero as we take smaller and smaller microstates
- However, too many clusters have a drawback with finite data: there might be
 - clusters with very few transitions (poor estimates of the transition matrix)
 - clusters with no transitions at all (this can break the ergodicity!)
- A good choice should balance between these two extremes

Markov state modeling

How to choose the hyperparameters?

- Lag τ has two clear limits:
 - τ too small: Markov assumption is violated
 - τ too large: the model can only resolve long time scales ($> \tau$), and number of samples we can use decreases
 - A way to choose the lag τ is by studying the results (e.g. the largest time scale τ_1) as a function of this parameter, and selecting it in a plateau
-  τ should be chosen in a range of values in which results are robust w.r.t. its variations

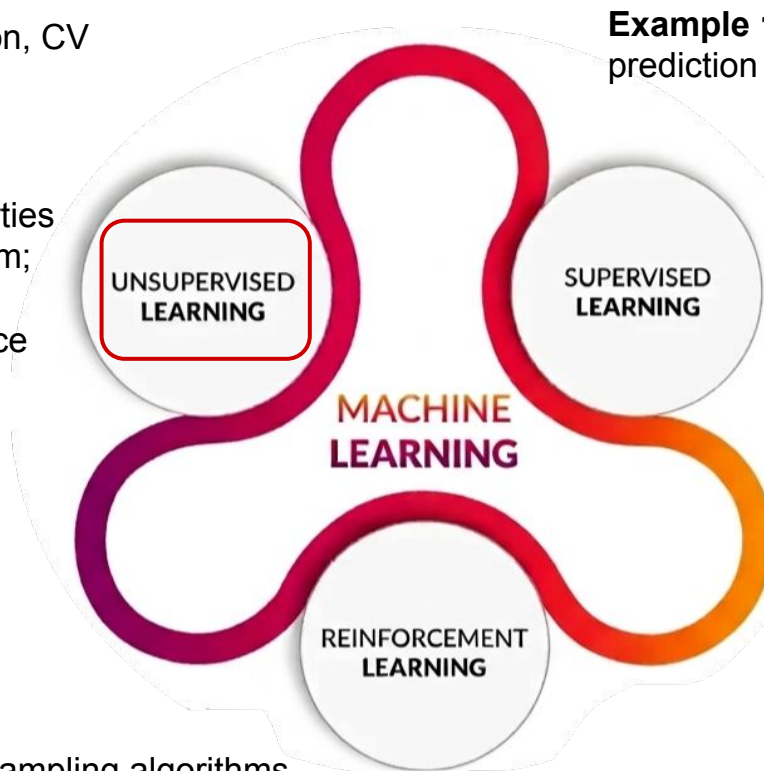
Backup slides

ML in molecular simulations

Examples: data visualization, CV construction, ...

↓
Goal: understanding properties of a high-dimensional system; learning variables that can describe it in a low-dim space

Examples: sampling algorithms, molecule design



Example 1: ML force-fields; graph NNs for prediction of electronic properties

↓
Goal: substituting Ab-initio MD / expensive QM calculations with cheaper tools, preserving the accuracy

Example 2: Selecting a subset of features in a candidate high-dimensional set of molecular descriptors

↓
Goal: reducing the dimensionality of the representation to make plots or use other analysis methods

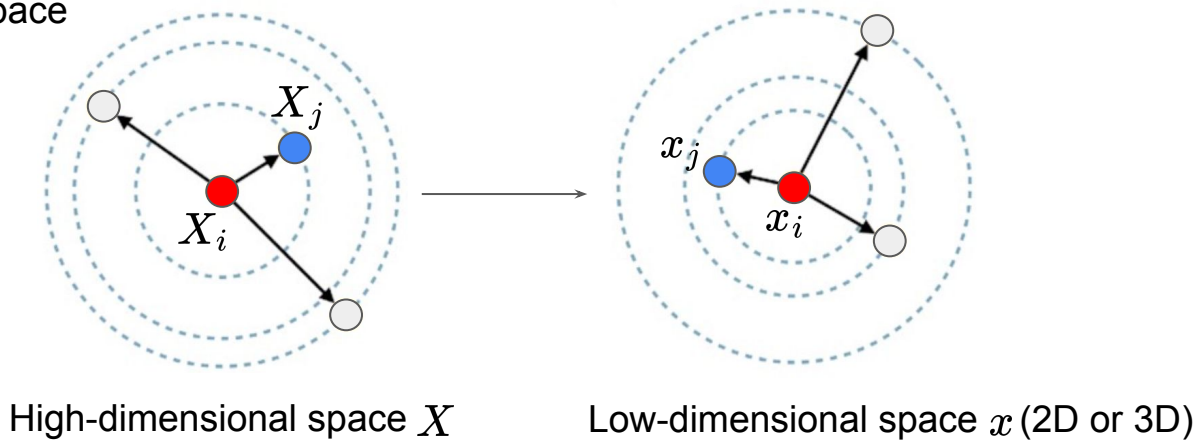
K-means vs K-medoids

Algorithm

- **Input:** cluster size k , data points $\{X_i\}$ or only distances $\{d(X_i, X_j)\}$
 - **Output:** cluster membership assignments $\{z_i\}$
1. Initialize k cluster medoids $\{M_l\}$
 2. Repeat until convergence of all $\{z_i\}$:
 - a) assign z_i based on the closest centroid: $z_i = \operatorname{argmin}_l d(X_i, C_l)$
 - b) update the medoids based on the memberships: $M_l = \operatorname{argmin}_{X_j} \sum_i \delta_{z_i, l} d(X_i, X_j)$

Stochastic Neighbor Embedding (SNE)

- **Idea:** Find a low-dimensional representation which preserves **local neighborhood structures** in the high-dimensional space



- First step: model the probability of point j being n.n. of i as a Gaussian in high-dim. space X :

$$P_i(j) = \frac{\exp\left(-\|X_i - X_j\|^2 / 2\sigma_i^2\right)}{\sum_{k(\neq i)} \exp\left(-\|X_i - X_k\|^2 / 2\sigma_i^2\right)}$$

σ_i : parameter of the method

normalized

Stochastic Neighbor Embedding (SNE)

- Second step: model the same probability in the low-dimensional space x :

$$Q_i(j) = \frac{\exp\left(-\|x_i - x_j\|^2\right)}{\sum_{k(\neq i)} \exp\left(-\|x_i - x_k\|^2\right)}$$

- Third step: **learn the target representations** $\{x_i\}$ such that the probability distributions $\{Q_i\}$ are close to the reference distributions $\{P_i\}$

by minimizing the loss function $\mathcal{L}(\{x_i\}) = \sum_i \text{KL}(P_i \parallel Q_i) = \sum_i \sum_{j(\neq i)} P_i(j) \log \frac{P_i(j)}{Q_i(j)}$

Stochastic Neighbor Embedding (SNE)

- Main limitation: **crowding problem**



In space X (high-dim.), a point i typically has several nearest neighbors at comparable distances: probabilities $P_i(j)$ are not sparse



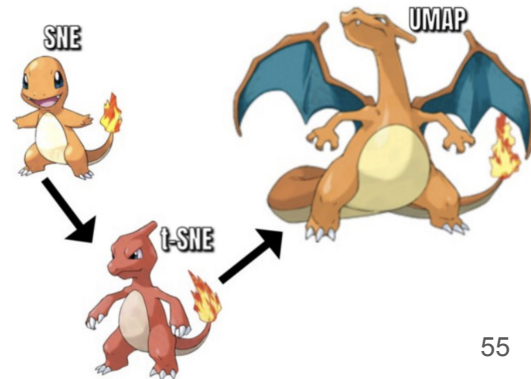
Trying to reproduce this situation in a low-dim. space makes the neighborhoods “overcrowded”, bringing to plots in which clusters are not well separated

- Solution (Van der Maaten and Hinton, 2008): substitute the Gaussian spread in space x with a distribution with **longer tails** (to push neighbors further and avoid crowding)



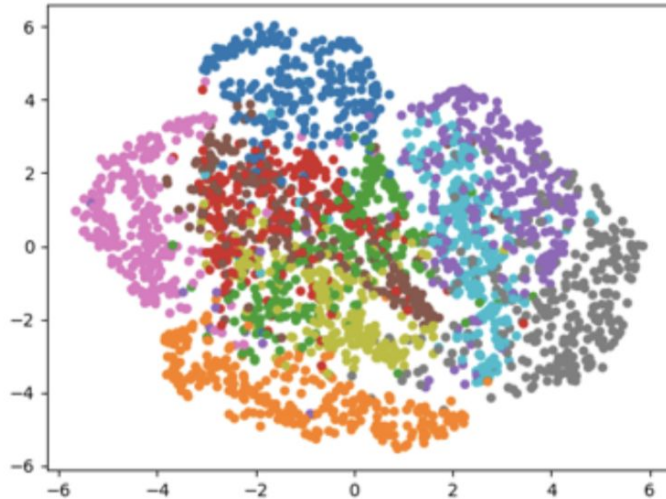
use t-distribution:
$$Q_i(j) = \frac{(1 + \|x_i - x_j\|^2)^{-1}}{\sum_{k(\neq i)} (1 + \|x_i - x_k\|^2)^{-1}}$$

- UMAP (McInnes, Healy, Melville, 2018) follows a similar philosophy (different distributions and loss) but it's faster!

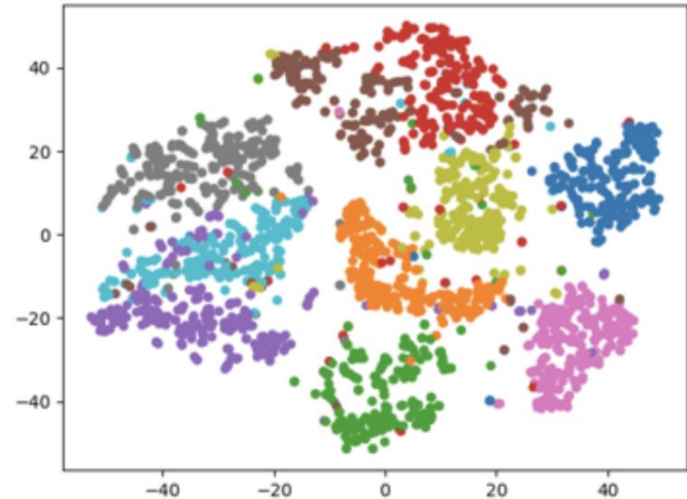


SNE vs t-SNE

SNE



t-SNE



Hierarchical clustering

Two types:

- **Agglomerative:**

- Start with the points as individual clusters
- At each step, **merge** the closest pair of clusters until one remains

- **Divisive:**

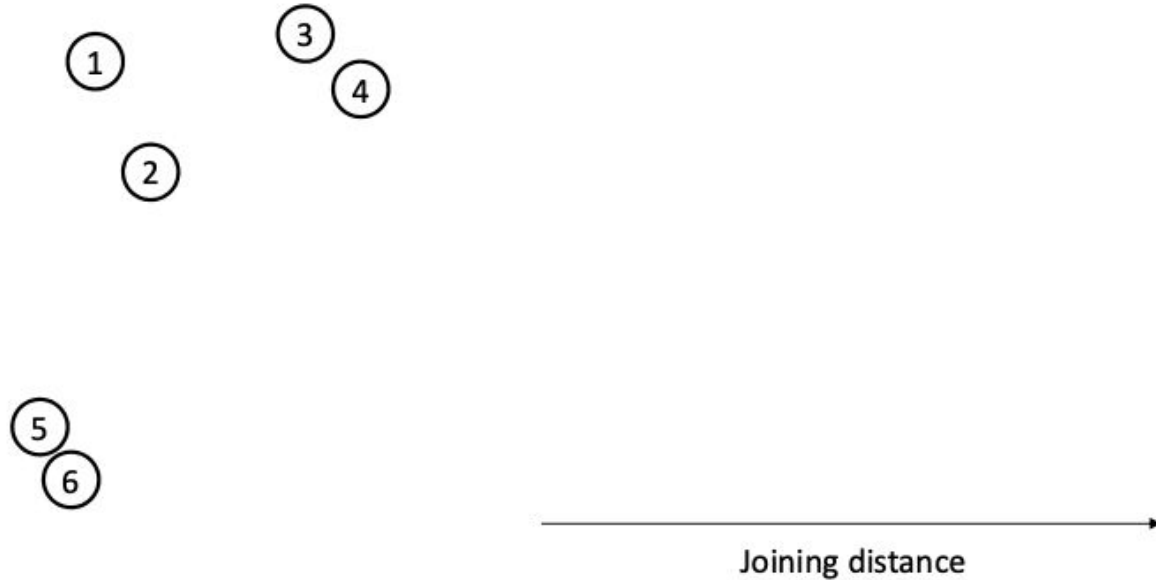
- Start with one, all-inclusive cluster
- At each step, **split** a cluster until each cluster contains a point

Agglomerative hierarchical clustering

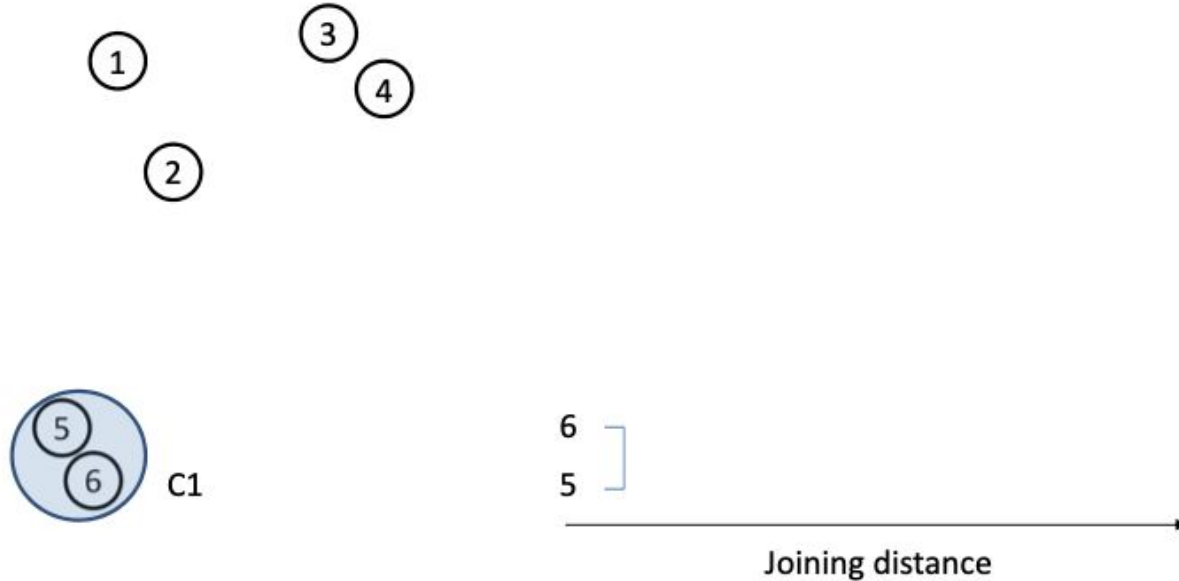
Basic algorithm:

1. Compute the distance matrix between the data points
2. Let each point be a cluster
3. Repeat until one cluster remains:
 - a. Merge the two closest clusters
 - b. Update the distance matrix

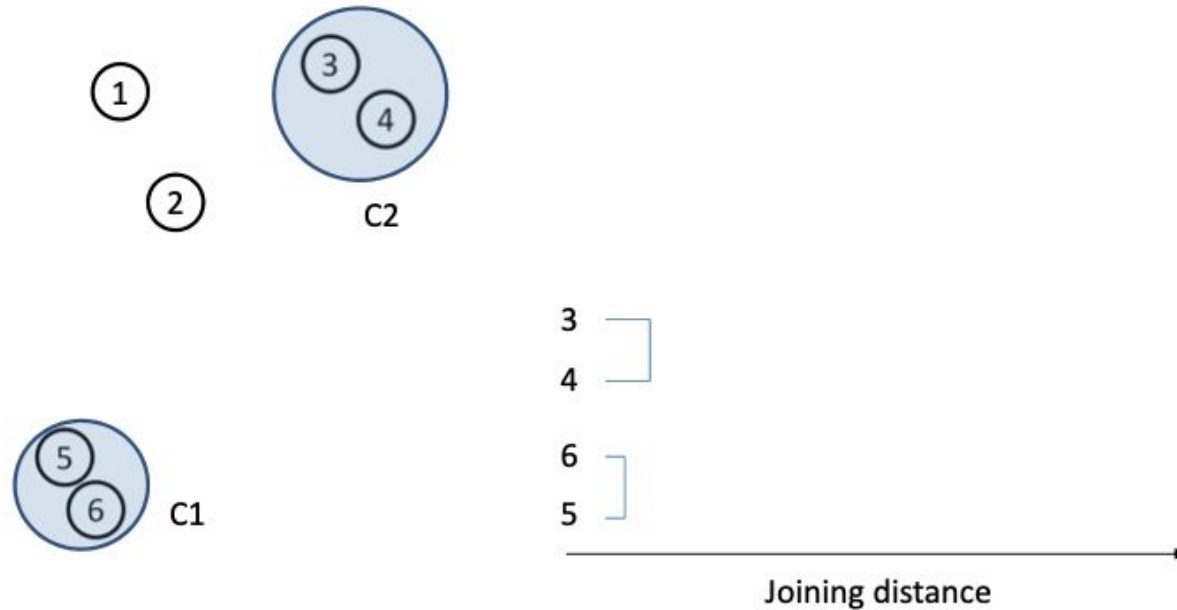
Agglomerative hierarchical clustering



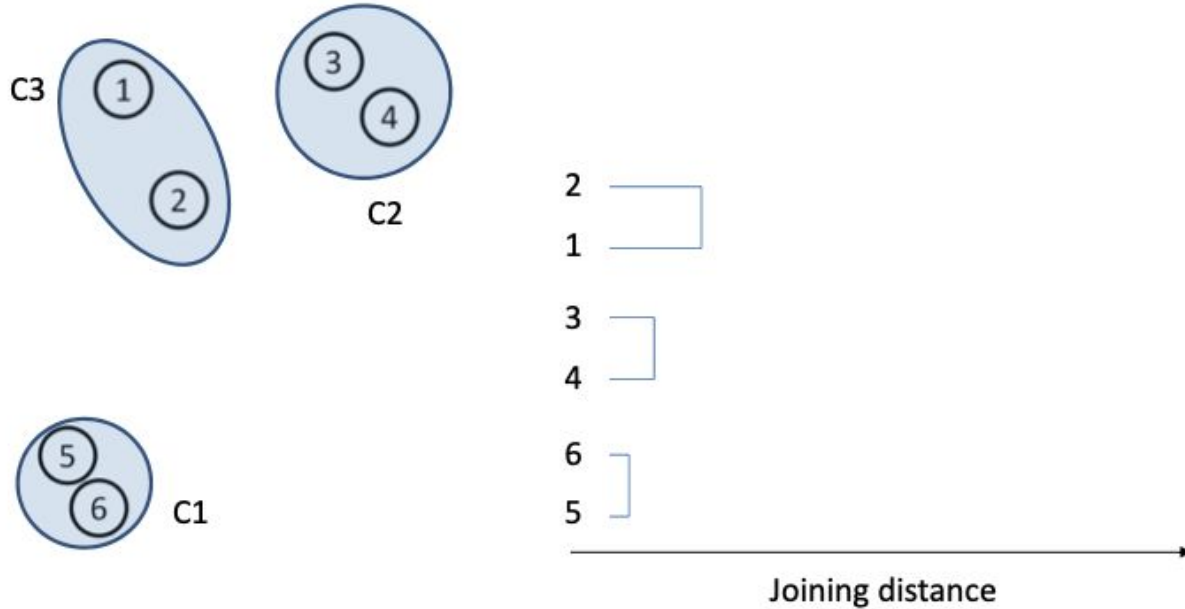
Agglomerative hierarchical clustering



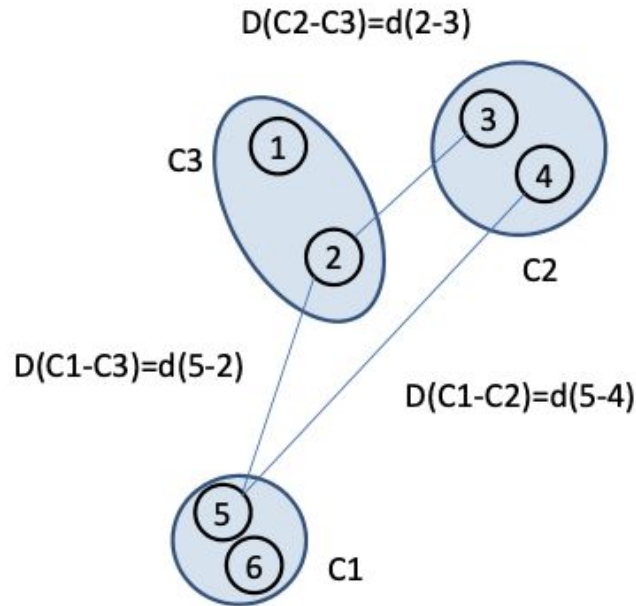
Agglomerative hierarchical clustering



Agglomerative hierarchical clustering

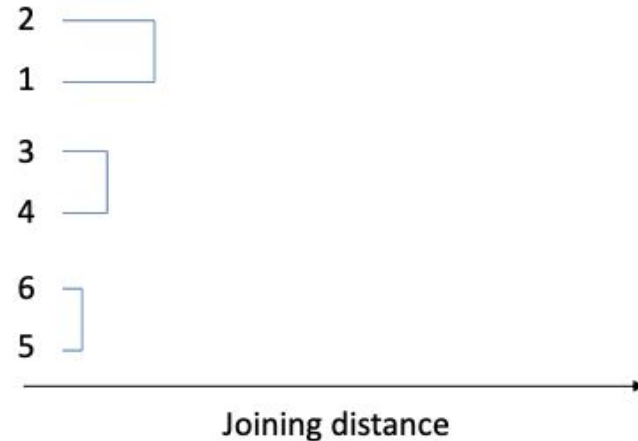


Agglomerative hierarchical clustering

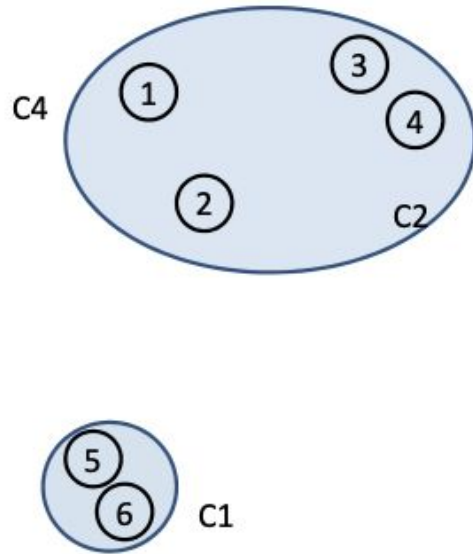


$$D(C_i, C_j) = \min\{d(X, Y) \text{ s.t. } X \in C_i, Y \in C_j\}$$

“Single-link distance”

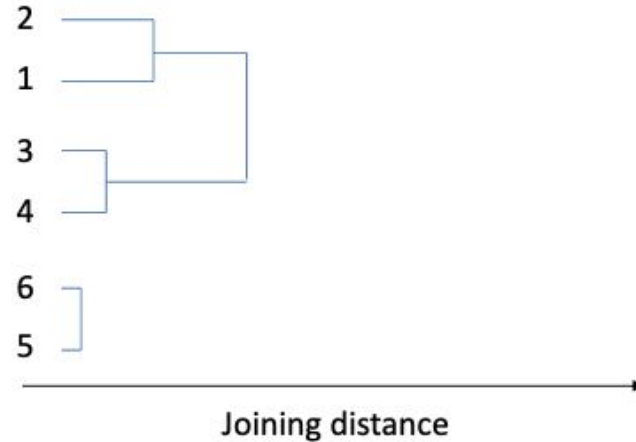


Agglomerative hierarchical clustering

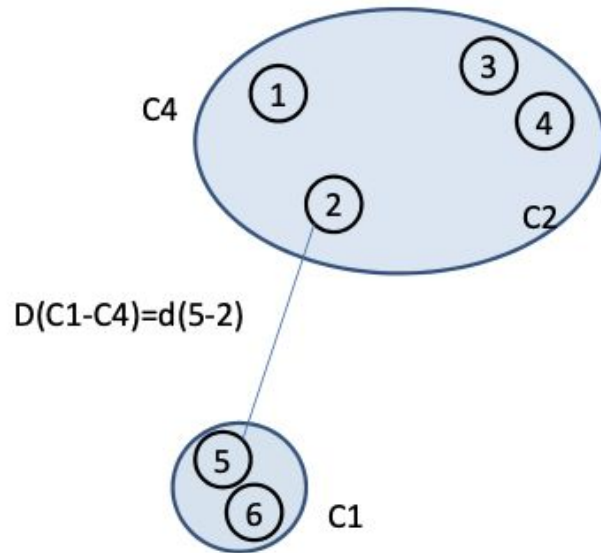


$$D(C_i, C_j) = \min\{d(X, Y) \text{ s.t. } X \in C_i, Y \in C_j\}$$

“Single-link distance”



Agglomerative hierarchical clustering



$$D(C_i, C_j) = \min\{d(X, Y) \text{ s.t. } X \in C_i, Y \in C_j\}$$

“Single-link distance”

